ENGINEERING REPORT & USER MANUAL

รายงานทางวิศวกรรมและคู่มือการใช้งาน

RP2040 CKP/CMP Signal Simulator



ภาพจำลองลักษณะตัวเครื่องภายนอกตามรายละเอียดการประกอบ

หน่วยงาน	P.U.K Car Service
ผู้จัดทำ	Engineer, P.U.K Car Service
แหล่งข้อมูล	ไฟล์โปรแกรม main(4).py และแผนผังวงจรที่แนบ
วันที่จัดทำ	20 สิงหาคม 2569

สาระสำคัญของระบบ

CKP/CMP Signal Simulator เป็นอุปกรณ์สร้างสัญญาณดิจิทัลเลียนแบบเซนเซอร์ตำแหน่งเพลาข้อเหวี่ยง (Crankshaft Position: CKP) และเซนเซอร์ตำแหน่งเพลาลูกเบี้ยว (Camshaft Position: CMP) โดยใช้ Raspberry Pi Pico/RP2040 ทำงานด้วย MicroPython และใช้ Programmable I/O (PIO) สร้างพัลส์ที่มีจังหวะสม่ำเสมอกว่าการสลับ GPIO ด้วยลูปโปรแกรมทั่วไป ผู้ใช้เลือก Engine Pattern ด้วยปุ่ม 3 ปุ่ม ปรับรอบด้วย KY-040 และดูสถานะบน OLED 1.3 นิ้ว

ขอบเขตที่ยืนยันจากโค้ด Pattern 6 แบบ, RPM 100–15,000, Step 10/100 RPM, OLED SH1106/SSD1306, ปุ่ม Active LOW 3 ปุ่ม, KY-040 แบบ Interrupt และเอาต์พุต Logic 3.3V ที่ GPIO14/15

วัตถุประสงค์และการนำไปใช้งาน

- ใช้เป็นแหล่งสัญญาณ CKP/CMP สำหรับการศึกษาและสาธิตหลักการทำงานของ ECU
- ใช้ตรวจสอบอุปกรณ์รับสัญญาณ ชุดทดลอง หรือวงจรอินเทอร์เฟซในห้องปฏิบัติการ
- ใช้ฝึกอ่าน Missing-tooth pattern, ความสัมพันธ์ CKP–CMP และผลของ RPM ต่อความถี่
- ใช้เป็นฐานพัฒนาเครื่องมือ Service/Training ของบริษัท และเพิ่ม Pattern ผ่านซอฟต์แวร์
- ใช้ทดสอบการแสดงผล การเลือก Pattern และ Human–Machine Interface แบบ Stand-alone

คำเตือนสำคัญ เอาต์พุตจากวงจรที่แนบเป็น Logic 3.3V จาก RP2040 ไม่ใช่สัญญาณเซนเซอร์ยานยนต์ที่รับรองแล้ว ห้ามต่อเข้ารถหรือ ECU โดยตรงจนกว่าจะตรวจสอบชนิดอินพุต ระดับแรงดัน การดึงขึ้น/ลง การป้องกันกระแสย้อน กราวด์ และวงจร Buffer/Isolation ให้เหมาะสม

1. ลักษณะทางกายภาพของต้นแบบ

ตัวเครื่องติดตั้งในกล่อง Future Box สีดำ วางอุปกรณ์บนแผงหน้าตามลำดับจากบนลงล่างเพื่อให้ผู้ใช้มองเห็นข้อมูลก่อนแล้วจึงเข้าถึงส่วนควบคุมและสวิตช์จ่ายไฟอย่างเป็นลำดับ

ลำดับ	องค์ประกอบบนตัวเครื่อง	หน้าที่
1	จอ OLED 1.3 นิ้ว I2C	แสดงหน้าเลือก Engine Pattern หรือหน้า RPM และค่า Step
2	ปุ่มกดสีแดง 3 ปุ่มเรียงแนวนอน	LEFT เลื่อนย้อนกลับ, CENTER ยืนยัน, RIGHT เลื่อนไปข้างหน้า
3	Rotary Encoder KY-040	หมุนปรับ RPM; กดแกนเพื่อสลับ Step 100/10 RPM
4	สวิตช์เปิด–ปิดสีแดง	ตัด–ต่อแหล่งจ่ายไฟของตัวเครื่อง; ไม่ใช่ Software RUN/STOP ในโค้ด
5	สายสัญญาณ CMP และ CKP 2 เส้น	นำ Logic Output 3.3V ออกจาก GPIO14 และ GPIO15
6	Adapter 12V 2A	จ่ายไฟเข้า DC–DC Buck Converter 12V→5V



รูปที่ 1 ภาพจำลองตัวเครื่อง: กล่องดำ จอ OLED ปุ่มแดง 3 ปุ่ม KY-040 สวิตช์แดง และสาย CMP/CKP

2. สถาปัตยกรรมฮาร์ดแวร์

2.1 เส้นทางพลังงาน

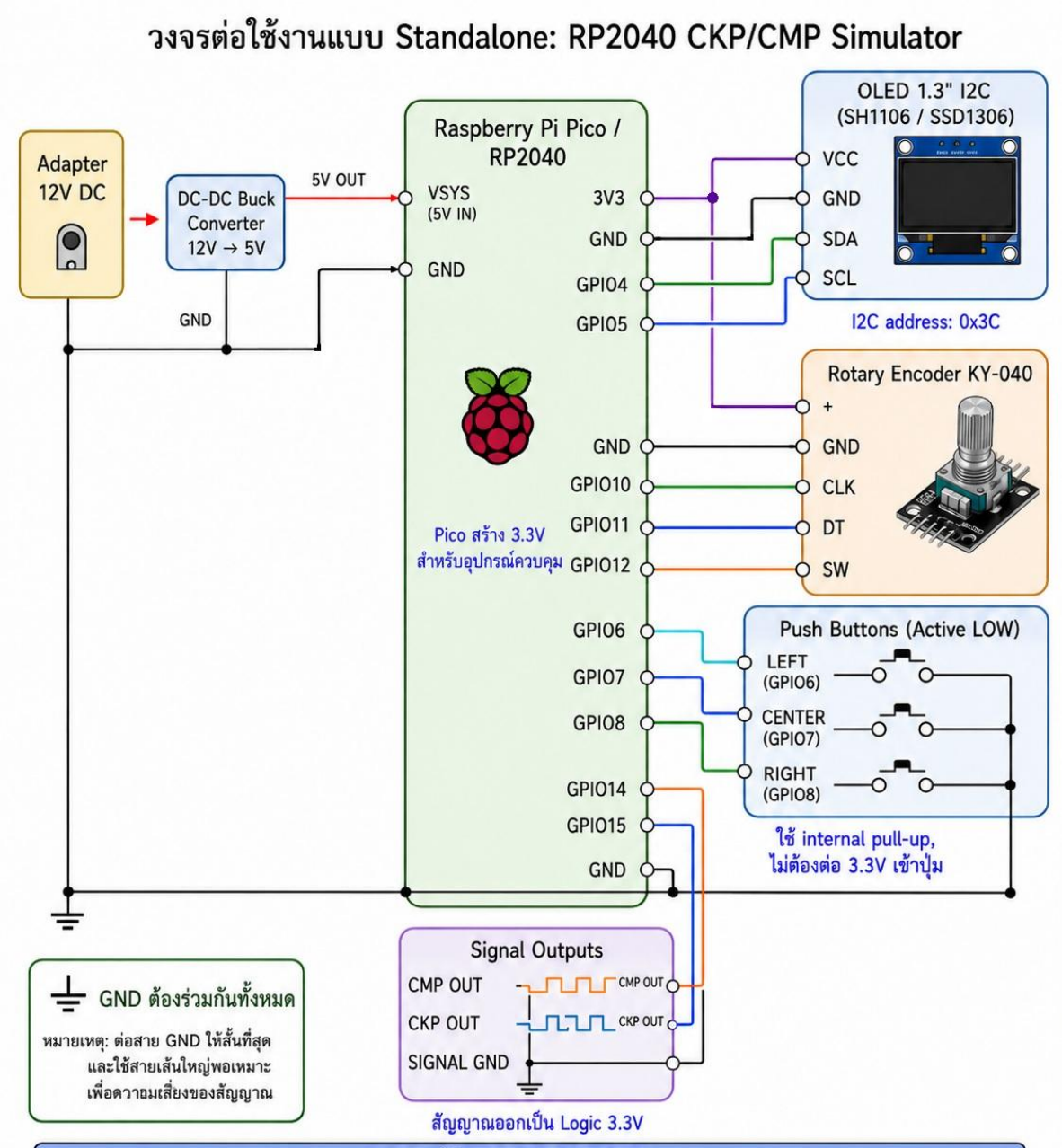
Adapter 12V DC 2A จ่ายไฟเข้าวงจร DC–DC Buck Converter ซึ่งลดแรงดันลงเป็น 5V แล้วป้อนเข้าขา VSYS ของ Raspberry Pi Pico/RP2040 ส่วนอุปกรณ์ควบคุม ได้แก่ OLED และ KY-040 ใช้ไฟ 3.3V จากบอร์ดเพื่อลดความเสี่ยงต่อ GPIO ของ RP2040 กราวด์ของแหล่งจ่าย บอร์ด OLED Encoder ปุ่ม และสัญญาณเอาต์พุตต้องต่อร่วมกันทั้งหมด

ก่อนเปิดเครื่องครั้งแรก ปรับ Buck Converter ให้ได้ 5.0V และวัดด้วยมัลติมิเตอร์ก่อนต่อเข้าขา VSYS ห้ามป้อน 12V เข้าบอร์ด RP2040 โดยตรง

2.2 ตารางกำหนดขา GPIO

GPIO	ชื่อสัญญาณ	อุปกรณ์/หน้าที่	ลอจิก
GPIO4	OLED SDA	I2C0 Data	3.3V
GPIO5	OLED SCL	I2C0 Clock	3.3V
GPIO6	LEFT	ปุ่มเลื่อน Pattern ย้อนกลับ	Active LOW / Pull-up
GPIO7	CENTER	ยืนยัน Pattern	Active LOW / Pull-up
GPIO8	RIGHT	ปุ่มเลื่อน Pattern ถัดไป	Active LOW / Pull-up
GPIO10	KY-040 CLK	Encoder Channel A	Pull-up / IRQ
GPIO11	KY-040 DT	Encoder Channel B	Pull-up / IRQ
GPIO12	KY-040 SW	สลับ RPM Step	Active LOW / Pull-up
GPIO14	CMP OUT	PIO output bit 0	Logic 3.3V
GPIO15	CKP OUT	PIO output bit 1	Logic 3.3V

2.3 แผนผังการต่อใช้งาน



สรุปการใช้งานขา (Pin Summary)	
GPIO4 = OLED SDA	GPIO10 = KY-040 CLK
GPIO5 = OLED SCL	GPIO11 = KY-040 DT
GPIO6 = LEFT	GPIO12 = KY-040 SW
GPIO7 = CENTER	GPIO14 = CMP OUT
GPIO8 = RIGHT	GPIO15 = CKP OUT

รูปที่ 2 แผนผังวงจร Stand-alone ที่ใช้เป็นข้อมูลอ้างอิงของรายงาน

3. ซอฟต์แวร์และไลบรารี

3.1 แพลตฟอร์มพัฒนา

โปรแกรมเขียนด้วย MicroPython สำหรับ RP2040 โดยไฟล์หลักคือ main(4).py เมื่อนำไปบันทึกบนบอร์ดในชื่อ main.py โปรแกรมจะเริ่มทำงานอัตโนมัติหลังเปิดเครื่องหรือ Reset เหมาะกับการทำงาน Stand-alone โดยไม่ต้องเชื่อมต่อคอมพิวเตอร์ระหว่างใช้งาน

โมดูล/ไลบรารี	แหล่งที่มา	หน้าที่ในระบบ
rp2	MicroPython port สำหรับ RP2040	ประกาศ PIO Assembly และสร้าง StateMachine
machine.Pin	MicroPython built-in	กำหนด GPIO, Pull-up และ Interrupt
machine.I2C	MicroPython built-in	สื่อสารกับ OLED ผ่าน I2C0 ที่ 400 kHz
time.ticks_ms	MicroPython built-in	อ่านเวลาสำหรับ Debounce และจำกัดอัตรา Refresh
time.ticks_diff	MicroPython built-in	คำนวณผลต่างเวลาแบบปลอดภัยเมื่อค่า Tick วนรอบ
sh1106.py	Driver ภายนอก ต้องมีบนบอร์ด	ควบคุม OLED SH1106 128×64 ที่ Address 0x3C
ssd1306.py	Driver ภายนอก/ทางเลือก	ใช้แทนเมื่อโมดูลจอเป็น SSD1306

ไฟล์ที่ต้องมีบนบอร์ด อย่างน้อยต้องมี main.py และ Driver ที่ตรงกับ OLED_DRIVER ในโค้ด เช่น sh1106.py เมื่อกำหนด OLED_DRIVER = "SH1106" หากไม่มีไฟล์ Driver โปรแกรมจะหยุดด้วย ImportError

3.2 ค่ากำหนด OLED

```
OLED_DRIVER = "SH1106"
OLED_WIDTH  = 128
OLED_HEIGHT = 64
OLED_ADDR   = 0x3C
I2C(0, sda=Pin(4), scl=Pin(5), freq=400_000)
```

โค้ดรองรับทั้ง SH1106 และ SSD1306 โดยเปลี่ยนค่า OLED_DRIVER แต่ต้องตรวจสอบ Controller ของจอจริงก่อน เนื่องจากจอขนาด 1.3 นิ้วจำนวนมากใช้ SH1106 ขณะที่บางรุ่นใช้ SSD1306

4. หลักการสร้างสัญญาณด้วย PIO

RP2040 มีหน่วย PIO ซึ่งทำงานแยกจาก CPU หลัก โปรแกรมใช้ State Machine 0 ที่ความถี่ 1,000,000 Hz หรือหนึ่งคำสั่งพื้นฐานประมาณ 1 µs โดยกำหนด out_base ที่ GPIO14 ทำให้การส่ง out(pins, 2) ควบคุม GPIO14–15 พร้อมกัน: bit 0 เป็น CMP และ bit 1 เป็น CKP

4.1 รูปแบบข้อมูล 32 บิตที่ส่งให้ PIO

ช่วงบิต	ข้อมูล	ความหมาย
bit 0–1	first_state	ระดับ CMP/CKP ในครึ่งแรกของ Slot

ช่วงบิต	ข้อมูล	ความหมาย
bit 2–3	second_state	ระดับ CMP/CKP ในครึ่งหลังของ Slot
bit 4–31	delay_count	จำนวนรอบหน่วง 28 บิต

```
return first_state | (second_state << 2) | (delay_count << 4)
```

State Machine ดึง Word จาก FIFO จากนั้นส่งสถานะแรกไปที่ GPIO หน่วงตามค่าที่กำหนด ส่งสถานะที่สอง และหน่วงอีกครั้ง วิธีนี้ทำให้ CKP และ CMP เปลี่ยนพร้อมกันบนขาที่ติดกันและลด Jitter จากงานแสดงผลหรือการอ่านปุ่ม

4.2 สูตรเวลาและความถี่

เวลาของหนึ่ง Tooth Slot คำนวณจาก slot_time_us = 60,000,000 ÷ (RPM × จำนวน Slot ต่อรอบ) ตัวอย่าง Pattern 60-2 ที่ 1,000 RPM จะมี Slot Time 1,000 µs และ Slot Frequency 1,000 Hz ส่วนที่ 15,000 RPM จะมี Slot Frequency 15,000 Hz

Pattern	Slot/รอบ	ความถี่ Slot ที่ 1,000 RPM	ความถี่ Slot ที่ 15,000 RPM
60-2	60	1,000 Hz	15,000 Hz
36-2 / 36-1	36	600 Hz	9,000 Hz
24-2 / 24-1	24	400 Hz	6,000 Hz
12+1	12 base slots	200 Hz	3,000 Hz

ข้อสังเกตการวัด มัลติมิเตอร์โหมด Hz อาจอ่านสัญญาณ Missing-tooth หรือ 12+1 ไม่ตรงกับค่าที่คำนวณ เพราะรูปคลื่นไม่ใช่ Square Wave ต่อเนื่อง ควรตรวจด้วย Oscilloscope หรือ Logic Analyzer และวัดทั้งความถี่ รูปแบบฟัน และเฟส CMP

5. ฐานข้อมูล Engine Pattern

Pattern ถูกเก็บใน Dictionary ชื่อ patterns และสามารถเพิ่มรายการใหม่ได้โดยกำหนดชื่อ จำนวน Slot ตำแหน่งฟันหายหรือฟันพิเศษ และช่วง CMP ในวัฏจักร 720 องศา โปรแกรมนับ cycle_slot ตั้งแต่ 0 ถึง (slots_per_rev × 2) − 1 ซึ่งเท่ากับสองรอบเพลาข้อเหวี่ยง

ID	ชื่อ	Slots/รอบ	ตำแหน่งพิเศษ	CMP HIGH (cycle_slot)
1	60-2	60	Missing 58, 59	10 ถึงก่อน 20
2	36-2	36	Missing 34, 35	6 ถึงก่อน 12
3	36-1	36	Missing 35	8 ถึงก่อน 14
4	24-2	24	Missing 22, 23	4 ถึงก่อน 8
5	24-1	24	Missing 23	4 ถึงก่อน 8
6	12+1	12	Extra tooth ที่ Slot 11	2 ถึงก่อน 4

5.1 การสร้าง Missing-tooth

เมื่อ tooth_position อยู่ใน missing_slots โปรแกรมส่งค่า CKP ระดับต่ำทั้ง first_state และ second_state ทำให้ไม่มีพัลส์ใน Slot นั้น แต่ CMP ยังคงรักษาระดับตามหน้าต่างที่กำหนด

5.2 การสร้าง Pattern 12+1

เมื่อถึง extra_tooth_slot โปรแกรมลดช่วงพัลส์เป็น quarter slot และส่ง Word เดิมเข้า FIFO สองครั้ง จึงเกิดพัลส์สองชุดภายในเวลาของ Slot พิเศษ เพื่อเลียนแบบฟันเพิ่มหนึ่งฟัน

การเพิ่ม Pattern ใหม่ ต้องตรวจรูปคลื่นอ้างอิงจริงจากผู้ผลิตหรือการวัดด้วย Oscilloscope ก่อนกำหนด missing, extra_tooth_slot และ cmp_start/cmp_end เพราะชื่อจำนวนฟันเพียงอย่างเดียวไม่ระบุเฟส CKP–CMP ของเครื่องยนต์แต่ละรุ่น

6. ส่วนติดต่อผู้ใช้และฟังก์ชันควบคุม

6.1 ปุ่มเลือก Pattern

- LEFT (GPIO6): ลด Pattern ID; เมื่อผ่านรายการแรกจะวนไป Pattern สุดท้าย
- RIGHT (GPIO8): เพิ่ม Pattern ID; เมื่อผ่านรายการสุดท้ายจะวนกลับ Pattern แรก
- CENTER (GPIO7): ยืนยัน browse_pattern_id ให้เป็น active_pattern_id และ Reset RPM เป็น 1,000

ปุ่มทั้งสามใช้ Internal Pull-up และทำงานแบบ Active LOW โดยมี Software Debounce 30 ms หน้าจอแสดง SELECTED หากรายการที่กำลังดูตรงกับ Pattern ที่กำลังสร้าง หรือแสดง CENTER = SET หากยังไม่ได้ยืนยัน

6.2 Rotary Encoder KY-040

Channel A และ B ใช้ Interrupt ทั้งขอบขาขึ้นและขาลง ตาราง QUADRATURE แปลงการเปลี่ยนสถานะ Gray Code เป็นทิศทาง เมื่อสะสมครบ 4 Edge จึงนับเป็นหนึ่ง Detent ช่วยกรอง Transition ที่ไม่ครบลำดับ การหมุนปรับ RPM และจำกัดค่าให้อยู่ระหว่าง 100 ถึง 15,000 RPM

สวิตช์กดบน Encoder ใช้ Debounce 30 ms และสลับ rpm_step ระหว่าง 100 RPM (Normal) กับ 10 RPM (Fine)

6.3 การแสดงผล OLED

หน้าจอ	สิ่งที่แสดง	เกิดขึ้นเมื่อ
ENGINE PATTERN	ชื่อ Pattern, ลูกศร, SELECTED/CENTER = SET	เปิดเครื่องหรือกด LEFT/RIGHT/CENTER
RPM	RPM ปัจจุบัน และ STEP 10/100	หมุนหรือกด Encoder

จอ Refresh เฉพาะเมื่อข้อมูลเปลี่ยน โดย service_display จำกัดช่วง Refresh ขั้นต่ำ 80 ms จึงลดภาระ I2C และลดการรบกวนลูปที่ป้อนข้อมูลเข้า PIO

6.4 สวิตช์แดงบนกล่อง

ข้อเท็จจริงจากโค้ด ไฟล์ main(4).py ไม่มี GPIO สำหรับ RUN/STOP และเริ่ม State Machine ทันทีหลัง Boot

ดังนั้นสวิตช์แดงตามตัวเครื่องควรตีความว่าเป็นสวิตช์ตัด–ต่อไฟหลัก เมื่อเปิดเครื่อง สัญญาณจะเริ่มทำงานโดยอัตโนมัติ

7. Tutorial: วิธีเตรียมโปรแกรมและติดตั้งลงบอร์ด

1. ติดตั้ง MicroPython

ติดตั้ง Firmware MicroPython สำหรับ Raspberry Pi Pico/RP2040 ลงบอร์ดตามเครื่องมือที่ใช้งาน เช่น Thonny หรือส่วนขยาย MicroPico ใน VS Code

2. เตรียมไฟล์

เปลี่ยนชื่อไฟล์ main(4).py เป็น main.py และเตรียม sh1106.py หรือ ssd1306.py ให้ตรงกับ OLED_DRIVER

3. คัดลอกลงบอร์ด

อัปโหลด main.py และไฟล์ Driver ไปยัง Root Filesystem ของบอร์ด ไม่ควรวางไว้เฉพาะในคอมพิวเตอร์

4. ตรวจ I2C

ก่อนใช้งานจริงควร Scan I2C และยืนยันว่าพบ Address 0x3C หากไม่พบให้ตรวจ SDA/SCL, VCC/GND และ Driver ของจอ

5. ทดสอบผ่าน USB

เปิด Serial/REPL ที่ Baud ตามเครื่องมือ ตรวจข้อความ Pattern selected, Pattern, RPM และ Step รวมทั้งแก้ ImportError หรือ I2C Error ก่อนประกอบลงกล่อง

6. ปิดไฟก่อนประกอบ

ถอด USB และ Adapter ก่อนเดินสายลง Future Box จากนั้นตรวจ Continuity และการลัดวงจรทุกจุด

ชื่อไฟล์สำคัญ MicroPython จะเรียก boot.py ก่อนและ main.py หลัง Boot หากไฟล์ยังชื่อ main(4).py จะไม่เริ่มอัตโนมัติแบบ Stand-alone เว้นแต่เครื่องมือ IDE จะสั่ง Run ให้

8. Tutorial: วิธีใช้งานตัวเครื่อง

1. ตรวจการต่อสาย

วางสวิตช์ที่ OFF ตรวจ Adapter 12V 2A, ขั้ว Buck Converter, สาย CMP/CKP และ Signal GND ห้ามให้ปลายสายสัมผัสกัน

2. ต่ออุปกรณ์วัดก่อน

สำหรับการทดสอบครั้งแรก ให้ต่อ CKP, CMP และ GND เข้ากับ Oscilloscope/Logic Analyzer ที่รับ 3.3V ได้ ไม่ควรเริ่มจาก ECU จริง

3. เปิดเครื่อง

ต่อ Adapter แล้วโยกสวิตช์แดงไป ON จอ OLED ควรแสดง ENGINE PATTERN และ Pattern เริ่มต้น 60-2 พร้อมคำว่า SELECTED

4. เลือก Engine Pattern

กดปุ่มแดงซ้ายหรือขวาเพื่อเลื่อนดู Pattern เมื่อได้รายการที่ต้องการให้กดปุ่มกลาง CENTER เพื่อยืนยัน ทุกครั้งที่ยืนยัน Pattern ค่า RPM จะกลับไป 1,000

5. ปรับ RPM

หมุน KY-040 เพื่อเพิ่มหรือลด RPM จอจะเปลี่ยนเป็นหน้า RPM ช่วงใช้งานตามโค้ดคือ 100–15,000 RPM

6. ปรับความละเอียด

กดแกน KY-040 เพื่อสลับ STEP: 100 สำหรับปรับเร็ว และ STEP: 10 สำหรับปรับละเอียด

7. ตรวจรูปคลื่น

ยืนยันจำนวนฟันหาย/ฟันพิเศษ คาบสัญญาณ Duty และเฟส CMP เทียบกับ CKP ตลอดช่วง RPM ที่ต้องการ

8. ปิดเครื่อง

โยกสวิตช์ OFF ก่อนถอดสายสัญญาณหรือเปลี่ยนการต่อวงจร รอให้จอดับแล้วจึงถอด Adapter

อาการบนจอ	ความหมาย	สิ่งที่ควรทำ
SELECTED	Pattern ที่แสดงกำลังถูกสร้างอยู่	หมุน Encoder หรือเริ่มตรวจสัญญาณ
CENTER = SET	กำลัง Browse แต่ยังไม่เปลี่ยน Output	กดปุ่มกลางเพื่อยืนยัน
STEP: 100	ปรับ RPM ครั้งละ 100	ใช้สำหรับไล่ช่วงเร็ว
STEP: 10	ปรับ RPM ครั้งละ 10	ใช้ Fine tuning
STOPPED	โปรแกรมได้รับ KeyboardInterrupt ผ่าน REPL	Reset/Power cycle เพื่อเริ่มใหม่

9. ขั้นตอนตรวจสอบและรับรองก่อนใช้งาน

รายการตรวจ	เกณฑ์เบื้องต้น	เครื่องมือ
Buck Output	5.0V คงที่ก่อนต่อ RP2040	Multimeter
3.3V Rail	อยู่ในระดับปกติของบอร์ด	Multimeter
I2C OLED	พบ 0x3C และแสดงผลครบ	REPL / OLED
ปุ่ม 3 ปุ่ม	เลื่อนและยืนยันได้ ไม่มี Double trigger	Functional test
Encoder	ทิศทางถูก Step 10/100 ทำงาน	Functional test
CKP Amplitude	Logic LOW/HIGH ใกล้ 0/3.3V	Oscilloscope
CMP Amplitude	Logic LOW/HIGH ใกล้ 0/3.3V	Oscilloscope
Pattern	จำนวน Slot/Missing/Extra ตรงฐานข้อมูล	Logic Analyzer
RPM Accuracy	ความถี่สัมพันธ์กับ RPM และ Slot	Oscilloscope calculation
Burn-in	ทำงานต่อเนื่องโดยไม่ Reset/ค้าง	1–2 ชั่วโมงตามแผน QC

10. ข้อจำกัดของซอฟต์แวร์เวอร์ชันปัจจุบัน

- ไม่มีการบันทึก Pattern, RPM หรือ Step ลง Flash; เปิดใหม่เริ่ม Pattern 60-2, RPM 1,000 และ Step 100

- ไม่มี Software RUN/STOP จากสวิตช์หน้าเครื่อง; State Machine เริ่มทันทีหลัง Boot
- ไม่มีเมนูตั้ง CMP Phase, Missing Tooth หรือจำนวน Slot จากหน้าเครื่อง; ต้องแก้ Dictionary ในโค้ด
- ไม่มีการ Scan I2C หรือข้อความ Error บนจอเมื่อ OLED/Driver ไม่พร้อม
- ไม่มี Feedback ยืนยันค่าความถี่จริงจากเอาต์พุต และไม่มี Self-test ของสาย CKP/CMP
- ไม่มี Watchdog, Error Log, Firmware Version บนจอ หรือระบบ Update ในตัว
- รูปคลื่นเป็น Push-pull Logic 3.3V; ยังไม่จำลองเอาต์พุต VR Sensor, Open-collector/Hall 5V หรือระดับ Automotive โดยตรง
- ช่วงสูงสุด 15,000 RPM เป็นค่าที่โค้ดยอมรับ แต่ต้องทดสอบ Accuracy/Jitter จริงในทุก Pattern ก่อนประกาศเป็นสเปกผลิตภัณฑ์

11. การแก้ปัญหาเบื้องต้น

อาการ	สาเหตุที่เป็นไปได้	แนวทางตรวจสอบ
จอไม่ติด	ไม่มี 3.3V, สลับ SDA/SCL, Driver ผิด, Address ไม่ใช่ 0x3C	วัดไฟ, Scan I2C, ตรวจ OLED_DRIVER และไฟล์ Driver
ขึ้น ImportError	ไม่มี sh1106.py/ssd1306.py บนบอร์ด	คัดลอก Driver ไป Root ของบอร์ด
ปุ่มกดไม่ตอบสนอง	ต่อปุ่มไม่ลง GND หรือ GPIO ผิด	ตรวจ GPIO6/7/8 และ Active LOW
Encoder กลับทิศ	CLK/DT สลับหรือกลไกคนละรุ่น	สลับ GPIO10/11 หรือแก้เครื่องหมาย Delta
RPM กระโดด	หน้าสัมผัส Encoder เด้ง/สายยาว/กราวด์ไม่ดี	ลดสาย, ตรวจ GND, เพิ่ม Hardware filter หากจำเป็น
ไม่มี CKP/CMP	GPIO14/15 ผิด, โปรแกรมหยุด, ไม่มี Common GND	ตรวจ Serial, Probe ที่ขาบอร์ด, ต่อ Signal GND
ความถี่ดูไม่ตรง	มิเตอร์อ่าน Missing-tooth ผิดวิธี	ใช้ Oscilloscope/Logic Analyzer และคำนวณ Slot frequency
เปลี่ยน Pattern แล้ว RPM กลับ 1000	เป็นพฤติกรรมที่กำหนดใน load_pattern()	ปรับ RPM ใหม่หลังยืนยัน Pattern

12. ข้อเสนอแนะสำหรับเวอร์ชันเชิงพาณิชย์

- เพิ่มปุ่มหรืออินพุต RUN/STOP แบบซอฟต์แวร์ แยกจากสวิตช์ไฟหลัก และบังคับเอาต์พุต LOW เมื่อ STOP
- เพิ่ม Output Stage เลือกได้ เช่น 3.3V Logic, 5V Open-collector และวงจรป้องกันสำหรับ ECU ตามสเปก
- เพิ่ม Fuse/Polyfuse, Reverse-polarity protection, TVS, Series resistor และ Isolation ตามระดับความเสี่ยง
- เพิ่มเมนูแสดง Firmware Version, Output status, Pattern, RPM, CKP/CMP frequency และ Warning
- บันทึกค่าล่าสุดแบบลด Wear หรือกำหนด Preset สำหรับงานบริการ
- เพิ่ม Test Point และขั้ว Signal GND ที่ชัดเจน พร้อม Connector ที่ป้องกันเสียบผิด
- จัดทำ Golden Waveform และ Automated QC สำหรับ Pattern ทุกแบบที่ RPM ต่ำ/กลาง/สูง
- ติด Serial Number, Revision, Pinout, Logic level และคำเตือนบนตัวเครื่อง

ภาคผนวก A: ลำดับการทำงานของโปรแกรม

ลำดับ	การทำงาน
1	ประกาศ PIO Program และกำหนด GPIO/ค่าคงที่
2	โหลด Pattern เริ่มต้น 60-2 และ Reset RPM เป็น 1,000
3	เริ่ม I2C และสร้าง OLED Object ตาม Driver
4	แสดงหน้า Pattern เริ่มต้น
5	ตั้งค่า Push Buttons และ Debounce State
6	ตั้งค่า KY-040 และ IRQ ทั้งสอง Channel
7	สร้าง StateMachine 0 ที่ 1 MHz และเปิดทำงาน
8	Loop: อ่านปุ่ม → อ่าน Encoder → คำนวณ Slot → สร้าง Word → ส่ง FIFO → Refresh OLED เมื่อจำเป็น
9	วน cycle_slot ครบสองรอบเพลาข้อเหวี่ยง แล้วกลับเป็นศูนย์
10	เมื่อ KeyboardInterrupt: ปิด PIO, ดึง CKP/CMP LOW และแสดง STOPPED

ภาคผนวก B: ค่าคงที่สำคัญใน main(4).py

ค่าคงที่	ค่า	ความหมาย
RPM_MIN	100	รอบต่ำสุด
RPM_MAX	15,000	รอบสูงสุดที่ซอฟต์แวร์จำกัด
RPM_STEP_NORMAL	100	Step ปกติ
RPM_STEP_FINE	10	Step ละเอียด
rpm เริ่มต้น	1,000	ค่าเริ่มและค่าหลังเลือก Pattern
PIO_FREQUENCY	1,000,000 Hz	Clock ของ State Machine
PIO_PATH_OVERHEAD	6	จำนวน Cycle ชดเชยเส้นทางคำสั่ง
BUTTON_DEBOUNCE_MS	30 ms	Debounce ปุ่ม 3 ปุ่ม
Encoder SW debounce	30 ms	Debounce ปุ่มกด Encoder
DISPLAY_MIN_INTERVAL_MS	80 ms	ช่วง Refresh จอต่ำสุด
OLED_ADDR	0x3C	I2C Address
I2C Frequency	400,000 Hz	ความเร็ว Bus

สรุป

ต้นแบบ CKP/CMP Signal Simulator นี้แสดงการประยุกต์ RP2040 PIO กับงานยานยนต์ได้อย่างเหมาะสมในระดับการทดลองและฝึกอบรม จุดเด่นคือการสร้างพัลส์แยกจากงาน UI รองรับ Pattern หลายแบบ ปรับ RPM ได้กว้าง และใช้งาน Stand-alone ด้วย OLED ปุ่ม และ Encoder อย่างไรก็ตาม ก่อนพัฒนาเป็นเครื่องมือสำหรับต่อ ECU จริง ต้องยืนยันรูปคลื่นอ้างอิง เพิ่มวงจรอินเทอร์เฟซ/ป้องกัน และจัดทำกระบวนการ QC ที่วัดได้ทุกเครื่อง

สถานะเอกสาร รายละเอียดฟังก์ชันซอฟต์แวร์ในรายงานอ้างอิงจาก main(4).py ที่ได้รับ
ส่วนลักษณะกล่องและตำแหน่งอุปกรณ์อ้างอิงจากคำอธิบายของผู้พัฒนาและภาพวงจรแนบ ภาพตัวเครื่องเป็นภาพจำลองประกอบเอกสาร ไม่ใช่ภาพถ่ายชิ้นงานจริง